EXPERIMENT 8: SHA-1 BRUTE-FORCE USING PYTHON

Task: T17

Name: Johan R Mathew

Roll Number: 17

1. Aim

To implement a SHA-1 brute-force program using Python's `hashlib` and `itertools`. The program generates candidate passwords from lowercase letters and digits, compares their SHA-1 hashes with a supplied test hash, and displays the password if found, the total number of attempts, and the execution time.

2. Course Outcome and Module

Course Outcome: Crack a student-generated, user-provided SHA-1 hash using a small alphanumeric character set and display the password if found.

Module: Cryptography and System Security.

3. Tools and Versions

Tool	Version	Operating System
Python 3.x	Record the exact version	Kali Linux
`hashlib`	Python standard library	Kali Linux
`itertools`	Python standard library	Kali Linux
`sha1sum`	Record version if required	Kali Linux

4. Program or Commands

Save the program as `sha1_crack.py`. The program checks the supplied hash, generates candidates of increasing length, and records attempts and execution time.

```
import hashlib
import itertools
import string
import sys
import time

def sha1_hash(password):
    return hashlib.sha1(password.encode("utf−8")).hexdigest()

def brute_force(target_hash, max_length=4):
    characters = string.ascii_lowercase + string.digits
    attempts = 0
    start = time.perf_counter()
    target_hash = target_hash.strip().lower()

    if len(target_hash) != 40 or any(c not in string.hexdigits for c in target_hash):
        print("Invalid SHA−1 hash.")
        return

    print("Starting SHA−1 brute−force test...")
```

```
    for length in range(1, max_length + 1):
        print(f"Trying passwords of length {length}...")
        for candidate_tuple in itertools.product(characters, repeat=length):
            candidate = "".join(candidate_tuple)
            attempts += 1
            if sha1_hash(candidate) == target_hash:
                elapsed = time.perf_counter() − start
                print(f"Password found: {candidate}")
                print(f"Total attempts: {attempts}")
                print(f"Execution time: {elapsed:.6f} seconds")
                return

    elapsed = time.perf_counter() − start
    print("Password not found within the search limit.")
    print(f"Total attempts: {attempts}")
    print(f"Execution time: {elapsed:.6f} seconds")

if __name__ == "__main__":
    if len(sys.argv) not in (2, 3):
        print("Usage: python3 sha1_crack.py <hash> [max−length]")
        sys.exit(1)
    target = sys.argv[1]
    maximum = int(sys.argv[2]) if len(sys.argv) == 3 else 4
    if not 1 <= maximum <= 5:
        print("Maximum length must be between 1 and 5.")
        sys.exit(1)
    brute_force(target, maximum)
```

5. How to Run

Open a terminal in Kali Linux and create the file `sha1_crack.py`. Paste the program above and save the file. Generate a test hash for `abc1` and run the program with that hash:

```
printf 'abc1' | sha1sum
python3 sha1_crack.py 9ee036287b4cfbcfa3b5bbfcf92d46eb5e75df96 4
```

Observe the password, attempt count, and execution time displayed by the program. The output shown below should be replaced or verified against the actual terminal run.

6. Sample Output with Screenshots

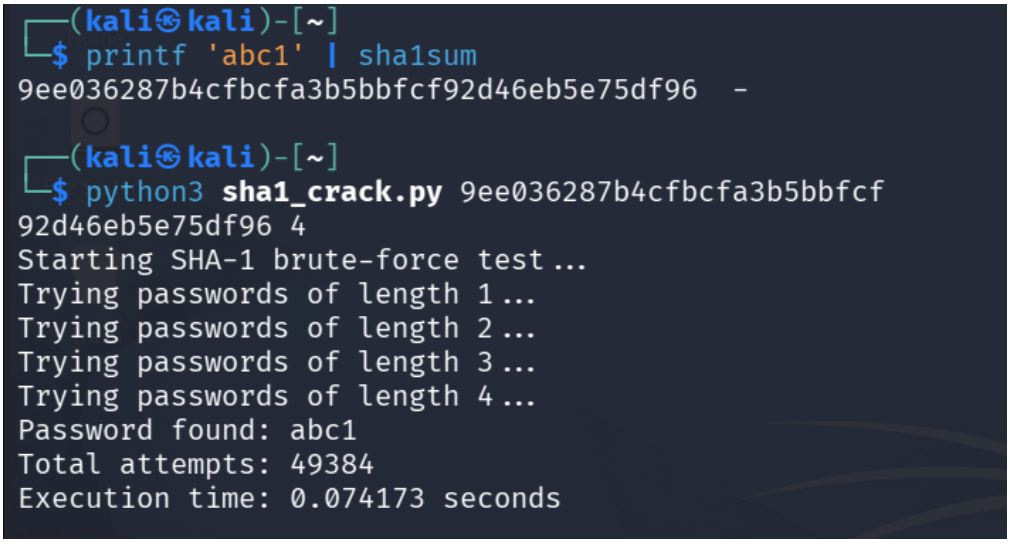


Figure T17-1: Python SHA-1 brute-force output

7. Result

The Python program generates candidate strings using `itertools.product`, computes SHA-1 digests using `hashlib`, and compares each digest with the target hash. For the test password `abc1`, a match should be found when the maximum candidate length is four. Record the actual attempt count and execution time from the terminal output.

8. Comparison with Other Versions

For an alphabet of 26 lowercase letters and 10 digits, there are 36 possible characters at each position. The number of candidates of length n is 36^n. The cumulative number of candidates from length one through length n is $\sum_{k=1}^{n} 36^k$. The search space therefore grows exponentially as the maximum length increases.

Length	Candidates at length	Cumulative candidates
1	36	36
2	1,296	1,332
3	46,656	47,988
4	1,679,616	1,727,604

Feature	T17: Python	T18: Bash
Language	Python	Bash shell script
Hash calculation	Python `hashlib` library	External `sha1sum` command
Candidate generation	`itertools.product`	Nested loops
Execution overhead	Hashes calculated within Python	External process is created for each candidate

Both implementations compare candidate hashes with a target hash. Actual speed depends on the implementation, hardware, and operating system.

9. Viva Questions and Answers

1. What is SHA-1? It is a hash function that produces a 160-bit digest, usually shown as 40 hexadecimal characters. It is not recommended for security-sensitive collision-resistant uses.

2. What is brute-force searching? It tests candidate values until a match is found or the search space is exhausted.

3. What is the purpose of `hashlib`? It provides functions for calculating hash digests, including SHA-1.

4. Why is `itertools.product` used? It generates character combinations for a specified candidate length.

5. Why does password length affect execution time? The number of candidates grows exponentially with length.

10. Common Errors: Message, Cause, and Fix

Message	**Cause**	**Fix**
Invalid hash	Hash is not 40 hexadecimal characters	Generate a valid SHA-1 test hash
Password not found	Password is outside the tested search space	Check the character set and maximum length
File not found	Wrong directory or filename	Run from the folder containing the program
Invalid command usage	Missing or incorrect arguments	Follow the displayed usage syntax

---

Declaration: AI tools were utilized for grammatical review, comparison with alternative versions, and assistance in preparing, reviewing, and verifying viva answers.